Project Report

AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)
FACULTY OF SCIENCE & TECHNOLOGY
SUMMER 2025 - 2026

Computer Graphics

Section: V
Group No: 03

Project Report on:

Fish Eating Game (2D)

Submitted to:

RAHNUMA TASMIN
Assistant Professor, Department of Computer Science
American International University-Bangladesh

Submitted By:

Name	ID	CONTRIBUTION (%)
Md. Naimul Haque Tashin	23-55346-3	25%
Rahul Roy	23-53662-3	25%
Tamjid Niloy	23-54109-3	25%
Jannate Ajmin Tanha	23-54420-3	25%

Date of Submission: 03-09-2026

1. Introduction

Computer graphics is not just about drawing static shapes on a screen it becomes far more interesting when those shapes are animated and made to respond to user input in real time. For our Computer Graphics course project, our team decided to build a small 2D interactive game called the Fish Survival Game, developed using C++ with OpenGL and the Free GLUT

The idea behind the game is simple and easy to understand, yet it allowed us to apply almost every major topic covered in the course. The game takes place inside an animated aquarium. The player controls an orange-colored player fish using the keyboard. Swimming around the aquarium are several colorful fish of different colours (red, yellow, green, blue and purple) which act as food the player earns points by guiding the orange fish into them. At the same time, dangerous enemy (monster) fish swim through the aquarium, and if one of them touches the player fish, the player loses one life.

The player starts with 3 lives (points). Every time the player is hit by an enemy fish, one life is lost and the orange fish is pushed back to its starting position with a short "hit" shake and a brief period of invulnerability so the player has a fair chance to recover. When all 3 lives are lost, the game ends and a Game Over screen is displayed along with the final score.

To keep the game engaging, it automatically becomes harder as the player's score increases. There are three difficulty levels Easy, Medium and Hard and the game moves from one level to the next once the score crosses a certain threshold. As the level increases, the enemy fish move faster, appear more frequently, and more of them can be on screen at the same time.

Although the game looks simple, building it required us to apply many core computer graphics concepts, including:

- 2D primitive drawing (polygons, triangles, quadrilaterals, lines and circles) using GL_POLYGON, GL_TRIANGLES, GL_QUADS, GL_LINES and GL_LINE_STRIP.
- Colour interpolation and shading using glColor3f/glColor4f per vertex (used heavily on the enemy fish body for a smooth gradient look).
- 2D transformations translation (glTranslatef) to move fish around the screen, and scaling (glScalef) to resize objects and to flip the player fish horizontally when it changes direction.
- Animation using time-based updates rather than fixed frame counts, driven by glutTimerFunc and a calculated deltaTime.
- Viewport and projection setup with gluOrtho2D and a custom reshape() function that preserves the aspect ratio of the game world.
- Alpha blending (glBlendFunc) for translucent overlays such as the level-up banner and the Game Over screen.
- Keyboard-based interactive input handling using GLUT's special-key and normal-key callbacks.
- Collision detection using axis-aligned bounding box (AABB) comparison between game objects.

In short, this project gave us the opportunity to combine graphics rendering, animation, and simple game logic into one complete, playable application.

2. Proposal

2.1 Proposed System

We propose a real-time, single-player 2D aquarium survival game rendered entirely with OpenGL primitives (no external image or texture files). The game world is a fixed-size aquarium (1100 × 720 units) containing an animated background, a controllable player fish, food fish, and enemy fish, along with a heads-up display (HUD) showing lives, score and current level.

2.2 Main Objectives

- To design and render 2D animated fish characters using only OpenGL primitive shapes.
- To implement smooth, physics-like player movement controlled by the keyboard.
- To implement a scoring system based on "eating" colourful fish.
- To implement a life/point system that is reduced when the player collides with an enemy fish.
- To implement three difficulty levels that automatically scale gameplay challenge as the score increases.
- To apply and demonstrate the core 2D computer graphics concepts taught in the course through a single working project.

2.3 Player Control

The player controls the orange fish using the arrow keys. The Up and Down keys move the fish vertically, and the Left and Right keys move it horizontally. Rather than moving the fish by a fixed number of pixels per key press, the game tracks which keys are currently held down and applies acceleration and deceleration every frame, which makes the fish feel like it has weight and momentum instead of moving in a robotic, snapped manner. The fish is also kept within the visible aquarium area by clamping its position between fixed minimum and maximum X/Y boundaries. When the player changes direction, the fish's visual model smoothly scales through zero on the X-axis to "flip" and face the new direction, instead of instantly popping to the mirrored image.

2.4 Scoring System (Colourful Fish)

Colourful food fish continuously spawn from the right edge of the screen and drift toward the left at a level-dependent speed. When the bounding box of the player fish overlaps the bounding box of a food fish, that food fish is deactivated (eaten) and the player's score increases by 10 points. After every score change, the game checks whether the score has crossed a level threshold and updates the difficulty level if needed.

2.5 Enemy Attacks and Point Reduction

Enemy (monster) fish also spawn from the right edge and move left, similar to food fish, but they are larger, visually threatening (dark colours, horns, sharp teeth) and are dangerous to the player. If the player fish's bounding box overlaps an enemy fish's bounding box and the player is not currently invulnerable, the player loses one life. The player fish is then reset to its starting position on the left side of the aquarium, plays a short shake animation, and becomes briefly invulnerable so it cannot lose multiple lives instantly from the same enemy. If the player's lives reach zero, the game enters the Game Over state.

2.6 Difficulty Levels

The game defines three difficulty levels Easy, Medium and Hard each with its own configuration for food speed, enemy speed, spawning frequency, and maximum number of active fish. The system automatically calculates the correct level from the current score:

- Easy: Score 0 – 199
- Medium: Score 200 – 399
- Hard: Score 400 and above

As the level increases, a short on-screen banner announces the new level, and a small burst of extra enemy fish is spawned immediately so the increased difficulty is noticeable right away rather than only after existing fish leave the screen.

2.7 Expected Learning Outcomes

Through this project, our team expected to strengthen our understanding of:

- How immediate-mode OpenGL drawing works and how complex shapes (fish, plants, stones, hearts) can be built from simple polygons and triangles.
- How to implement smooth, frame-rate-independent animation using delta time.
- How basic 2D transformations (translation and scaling) are used to position, animate, and mirror objects.
- How simple game logic (scoring, lives, levels, collision) is structured and connected to a rendering loop.
- How to work as a team on a single shared codebase, dividing work by feature area and later integrating everyone's code into one project.

3. Schematic Diagram

The following screenshots show the game running at different stages: normal gameplay at the Easy difficulty level, a later moment in the same run, gameplay at the Medium and Hard difficulty levels, and the Game Over screen displayed once all lives are lost.

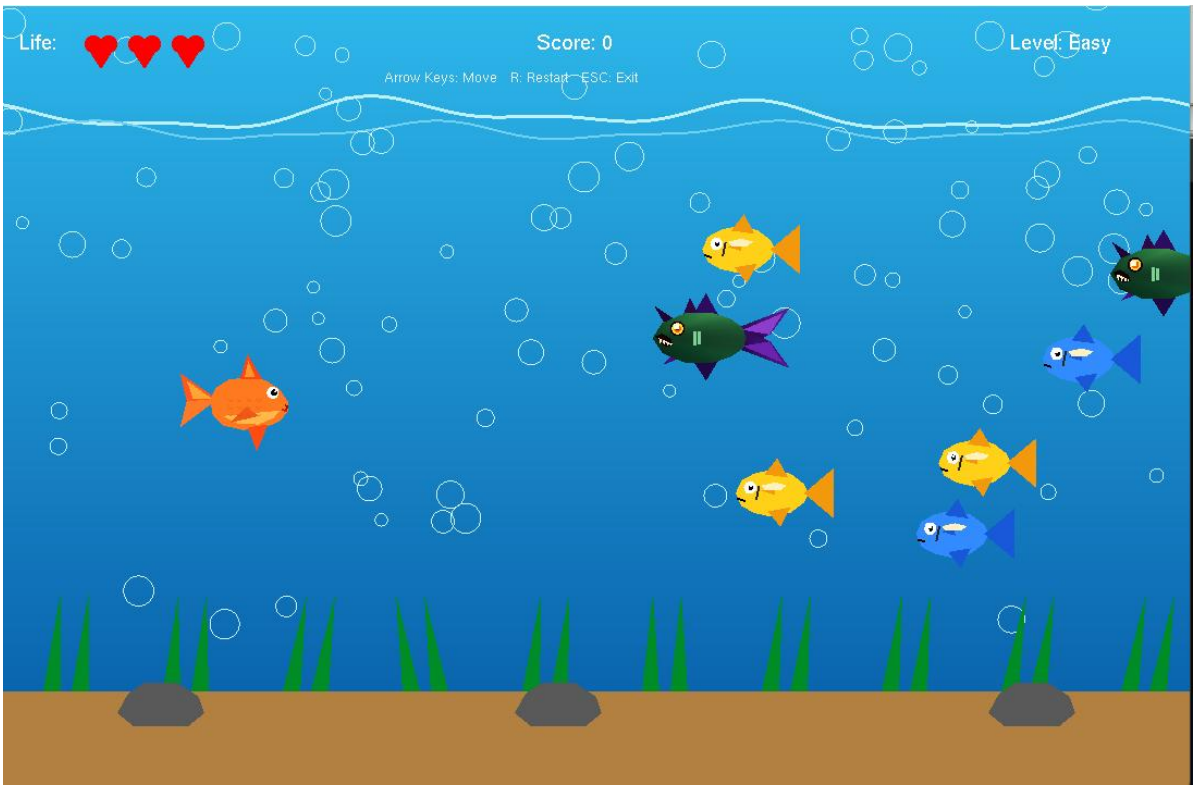

Figure 3.1 Gameplay at the start of a run (Score: 0, Level: Easy, 3 lives remaining).

Figure 3.2 Gameplay in progress (Score: 40) showing the player fish among colourful food fish and enemy fish.

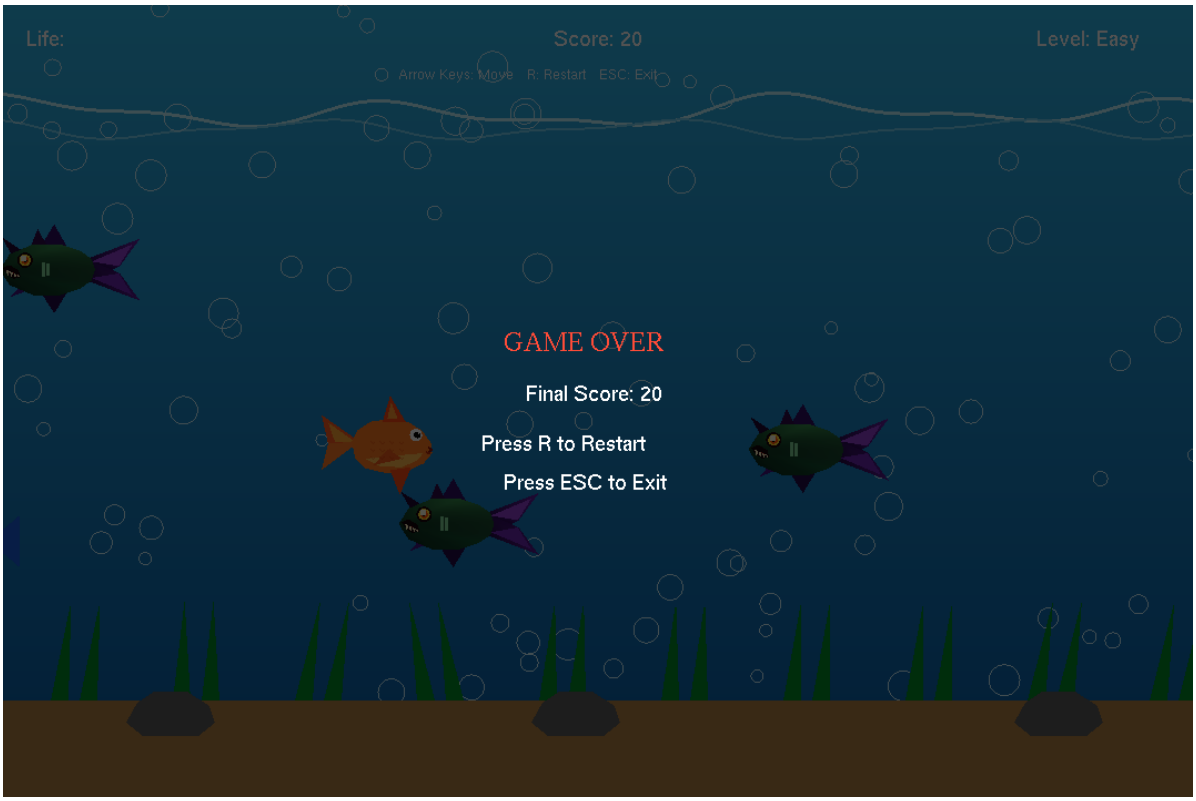

Figure 3.3 The Game Over overlay shown after all 3 lives are lost, with the final score and restart or exit instructions.

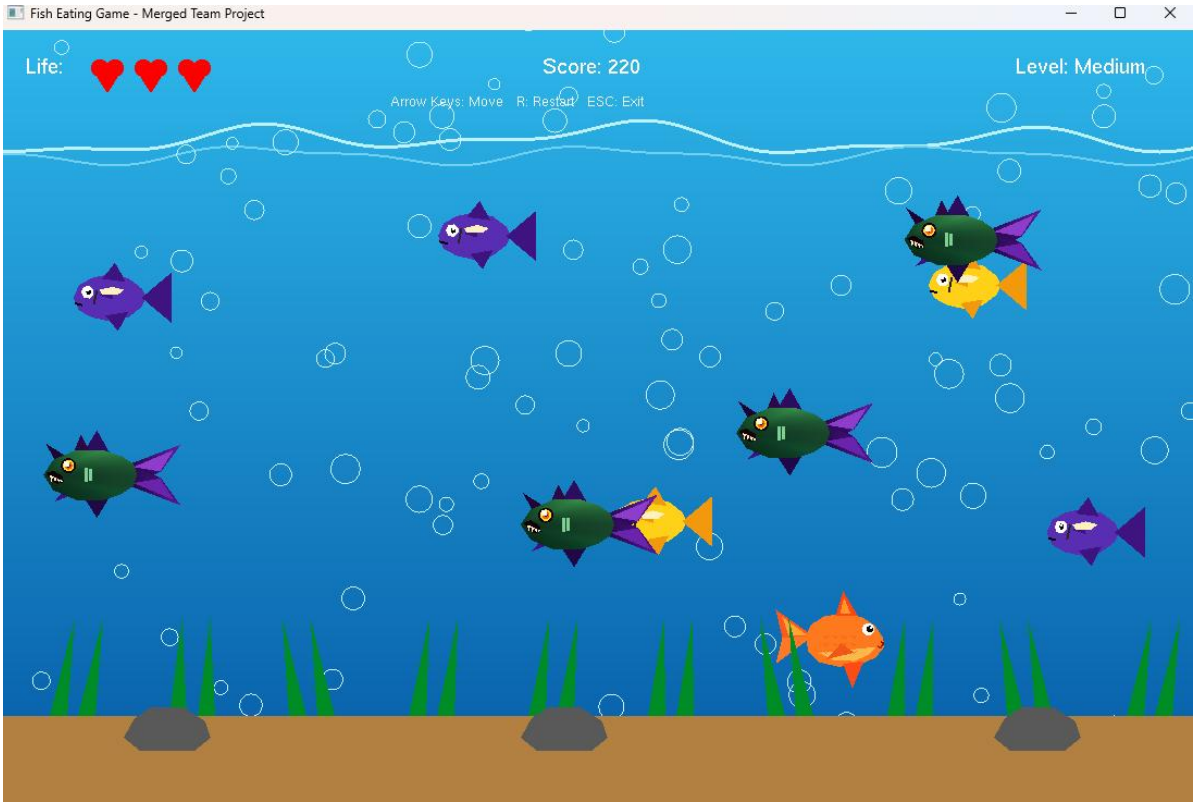

Figure 3.4 Gameplay at the Medium difficulty level (Score: 220, 3 lives remaining), showing multiple food and enemy fish on screen at once.

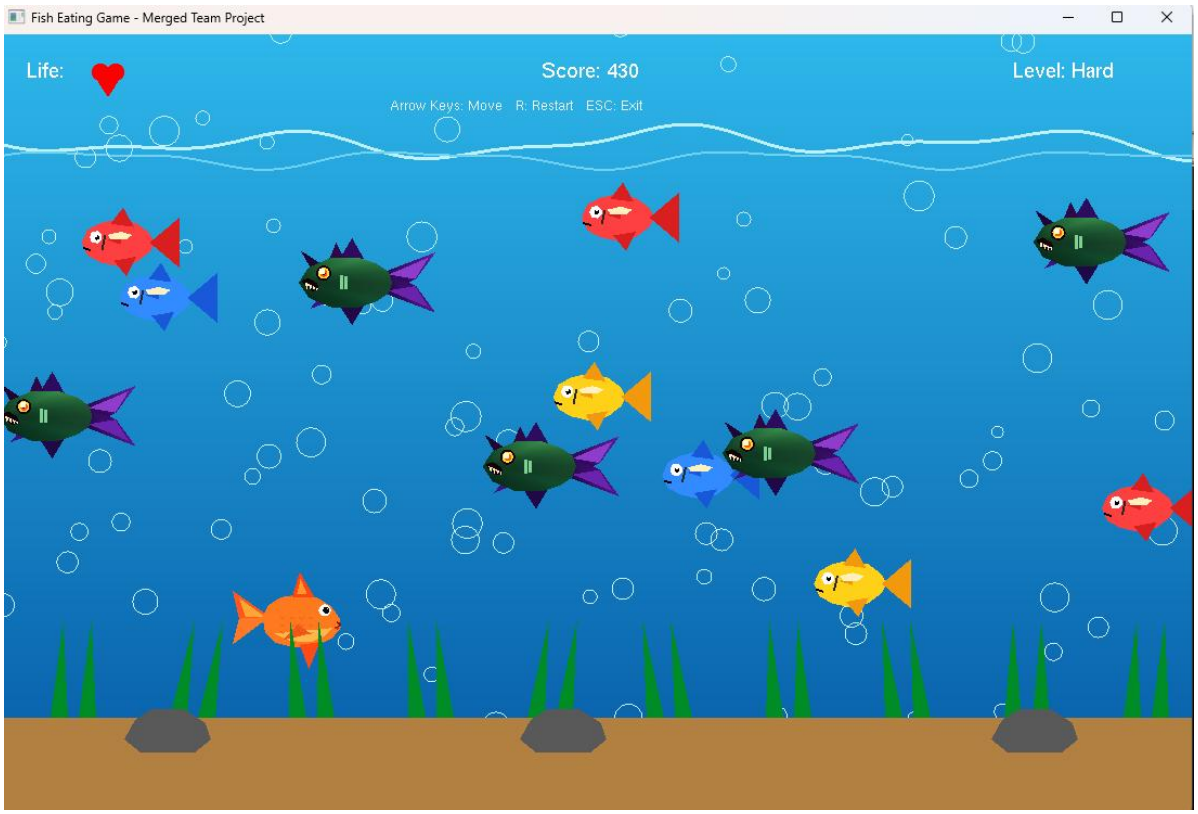

Figure 3.5 Gameplay at the Hard difficulty level (Score: 430, 1 life remaining), showing the increased number of enemy fish on screen.

4. List of Objects

Object Name	Description	Purpose
Player Fish	An orange fish built from multiple coloured polygons/triangles (body, belly, fins, tail, eye, mouth)	Represents the character controlled by the player using arrow keys
Colorful Fish (Food Fish)	Small fish in 5 colours (red, yellow, green, blue, purple) that swim from right to left	Acts as "food"; increases the player's score when eaten
Enemy Fish (Monster Fish)	A larger, dark-coloured fish with horns, sharp teeth and glowing eyes	Threat object; reduces the player's lives on collision
Background / Water	A gradient-coloured quad (light blue at top, dark blue at bottom) filling the screen	Provides the aquarium's underwater setting
Water Waves	Two animated line strips near the top of the screen using layered sine functions	Simulates the movement of the water surface
Bubbles	Circles of varying size that rise from the bottom and drift left/right	Adds visual realism and ambient motion to the aquarium
Soil (Aquarium Floor)	A brown rectangle at the bottom of the screen	Represents the sandy floor of the aquarium

Object Name	Description	Purpose
Grass	Green triangular blades that sway back and forth using a sine wave	Decorative aquarium plant life
Stones	Grey polygon shapes placed at fixed positions on the floor	Decorative aquarium elements
Life / Point Display (Hearts)	Red heart icons drawn using two circles and a triangle	Shows the player's remaining lives (starts at 3)
Score Display	Text drawn with drawText() showing the current score	Informs the player of their current score
Level Display	Text showing the current difficulty (Easy/Medium/Hard)	Informs the player which difficulty they are currently facing
Level-Up Message	A semi-transparent banner with text shown briefly after leveling up	Notifies the player that the difficulty has increased
Control Hint Text	Small text showing "Arrow Keys: Move, R: Restart, ESC: Exit"	Reminds the player of the available controls
Game Over Screen	A dark, semi-transparent full-screen overlay with final score and restart instructions	Displayed when the player's lives reach zero

5. Functions to Represent the Objects

Each visible game object is drawn using its own dedicated function. This kept our code organized and made it easy for different team members to work on different fish/objects without conflicts.

5.1 DrawPlayerFish()

Draws the orange player fish using a series of GL_POLYGON and GL_TRIANGLES calls for the body, belly, fins, tail and eye, all defined in local/normalized coordinates around the origin. drawPlayerFish() then positions the fish in the world using glTranslatef, applies the hit-shake offset, and uses glScalef (with a horizontal scale that can smoothly go from +1 to −1) to flip the fish when it changes direction.

```
void drawPlayerFish() {
    float shakeX = 0.0f, shakeY = 0.0f;
    if (player.shakeTime > 0.0f) {
        shakeX = std::sin(gameTime * PLAYER_SHAKE_X_FREQUENCY) * PLAYER_SHAKE_X_AMOUNT;
        shakeY = std::cos(gameTime * PLAYER_SHAKE_Y_FREQUENCY) * PLAYER_SHAKE_Y_AMOUNT;
    }
    glPushMatrix();
    glTranslatef(player.x + shakeX, player.y + shakeY, 0.0f);
    glScalef(player.facingScaleX, 1.0f, 1.0f);   // flips fish left/right
    glScalef(PLAYER_VISUAL_SCALE, PLAYER_VISUAL_SCALE, 1.0f);
    drawPlayerFishLocal();
    glPopMatrix();
}
```

5.2 DrawFoodFish()

Draws a colourful food fish. The body colour and fin colour are chosen from a small palette using setFoodFishColor() and setFoodFinColor(), so the same drawing function can produce all five fish colours. drawFoodFish() adds a small vertical "bobbing" motion using a sine function so the fish do not swim in a perfectly straight line.

```
void drawFoodFish(const FoodFish& fish) {
    float bob = std::sin(gameTime * FOOD_BOB_SPEED + fish.swimPhase) * FOOD_BOB_AMOUNT;
    glPushMatrix();
    glTranslatef(fish.x, fish.y + bob, 0.0f);
    glScalef(FOOD_VISUAL_SCALE, FOOD_VISUAL_SCALE, 1.0f);
    drawFoodFishLocal(fish.color);
    glPopMatrix();
}
```

5.3 DrawEnemyFish()

Draws the enemy/monster fish. This function makes heavy use of per-vertex colour interpolation (different glColor3f calls between vertices of the same polygon) to create a smooth colour gradient across the enemy's body and tail, which visually separates it from the simpler, flat-coloured food fish.

```
void drawEnemyFish(const EnemyFish& fish) {
    float bob = std::sin(gameTime * ENEMY_BOB_SPEED + fish.swimPhase) * ENEMY_BOB_AMOUNT;
    glPushMatrix();
    glTranslatef(fish.x, fish.y + bob, 0.0f);
    glScalef(ENEMY_VISUAL_SCALE, ENEMY_VISUAL_SCALE, 1.0f);
    drawEnemyFishLocal();
    glPopMatrix();
}
```

5.4 drawWater(), drawWave(), drawBubbles()

drawWater() fills the screen with a colour-interpolated quad to create a top-to-bottom gradient. drawWave() draws two animated GL_LINE_STRIP waves near the surface by summing multiple sine waves of different frequency and phase. drawBubbles() draws each bubble as a GL_LINE_LOOP circle using the shared drawCircle() helper.

```
void drawWater() {
    glBegin(GL_QUADS);
    glColor3f(0.18f, 0.72f, 0.92f); glVertex2f(0, HEIGHT); glVertex2f(WIDTH, HEIGHT);
    glColor3f(0.02f, 0.35f, 0.65f); glVertex2f(WIDTH, 0);  glVertex2f(0, 0);
    glEnd();
}
```

5.5 drawSoil(), drawGrass(), drawStones()

drawSoil() draws a simple brown rectangle for the floor. drawGrass() draws pairs of triangular leaves whose tip position sways using a sine function, giving the appearance of grass moving underwater. drawStones() draws three grey, hand-shaped polygons at fixed positions along the floor.

5.6 drawLives(), drawScore(), drawLevel()

These three functions form the main HUD. Each one builds a short text string (using sprintf for the score and level) and calls the shared drawText() helper, which uses glutBitmapCharacter() to render bitmap text at a given screen position. drawLives() additionally calls drawHeart() once per remaining life.

```
void drawScore() {
    char text[40];
    std::sprintf(text, "Score: %d", gameState.score);
    glColor3f(1.0f, 1.0f, 1.0f);
    drawText(HUD_SCORE_X, HUD_TOP_Y, text);
}
```

5.7 drawGameOverOverlay(), drawLevelMessage()

Both functions draw a translucent GL_QUADS rectangle over part (or all) of the screen using glColor4f with an alpha value below 1.0, which requires blending to be enabled (glEnable(GL_BLEND)). Text is then drawn on top to show the relevant message ("GAME OVER" / "Level! Speed Increased").

6. Interactive Functions

This section covers every function responsible for reading player input and turning it into gameplay changes.

6.1 Keyboard Controls — Arrow Keys (Movement)

Arrow key state is tracked with GLUT's special-key callbacks rather than moving the fish a fixed distance per key press. This avoids the operating system's key-repeat delay and produces smooth, continuous, frame-based motion.

```
void specialKeyDown(int key, int, int) {
    if (key == GLUT_KEY_LEFT)  { inputState.left  = true; player.direction = -1; }
    else if (key == GLUT_KEY_RIGHT) { inputState.right = true; player.direction = 1; }
    else if (key == GLUT_KEY_UP)    { inputState.up    = true; }
    else if (key == GLUT_KEY_DOWN)  { inputState.down  = true; }
}

void specialKeyUp(int key, int, int) {
    if (key == GLUT_KEY_LEFT)  inputState.left  = false;
    else if (key == GLUT_KEY_RIGHT) inputState.right = false;
    else if (key == GLUT_KEY_UP)    inputState.up    = false;
    else if (key == GLUT_KEY_DOWN)  inputState.down  = false;
}
```

6.2 Keyboard Controls — Restart and Exit

Normal (non-special) keys are handled by keyboard(). Pressing R calls restartGame() to reset the score, lives, level and all fish, and pressing ESC exits the program.

```
void keyboard(unsigned char key, int, int) {
    if (key == 27) std::exit(0);              // ESC
    if (key == 'r' || key == 'R') {
        restartGame();
        glutPostRedisplay();
    }
}
```

6.3 Player Movement — updatePlayerMovement()

This function converts the held arrow keys into a direction vector, applies acceleration/deceleration toward a target speed, updates the player's position, and clamps it within the aquarium boundaries.

```
player.velocityX = moveTowards(player.velocityX, targetVelocityX, horizontalRate * deltaTime);
player.velocityY = moveTowards(player.velocityY, targetVelocityY, verticalRate * deltaTime);
player.x += player.velocityX * deltaTime;
player.y += player.velocityY * deltaTime;
player.x = clampFloat(player.x, PLAYER_MIN_X, PLAYER_MAX_X);
player.y = clampFloat(player.y, PLAYER_MIN_Y, PLAYER_MAX_Y);
```

6.4 Enemy / Food Fish Movement — updateFoodFish(), updateEnemyFish()

Both functions move every active fish leftward at a speed defined by the current difficulty level, deactivate fish that leave the screen, and spawn new fish once their respective spawn timers reach zero.

6.5 Collision Detection — rectanglesOverlap()

All collisions (player–food and player–enemy) use a shared Axis-Aligned Bounding Box (AABB) test.

```
bool rectanglesOverlap(float ax, float ay, float aHalfW, float aHalfH,
                       float bx, float by, float bHalfW, float bHalfH) {
    return std::fabs(ax - bx) <= (aHalfW + bHalfW) &&
           std::fabs(ay - by) <= (aHalfH + bHalfH);
}
```

6.6 Score Update — handleFoodCollisions()

Checks every active food fish against the player's bounding box; on overlap, deactivates the fish and adds 10 points to the score, then re-checks the difficulty level.

6.7 Life Reduction — handleEnemyCollisions() / handlePlayerHit()

Checks every active enemy fish against the player's bounding box (only while the player is not invulnerable). On a hit, handlePlayerHit() decreases lives by one; if lives reach zero, the game enters the Game Over state, otherwise the player is respawned at the starting position with a short shake animation and temporary invulnerability.

6.8 Difficulty-Level Update — updateLevelFromScore()

Automatically recalculates the difficulty level from the current score and, when the level changes, shows the level-up banner and immediately spawns a small burst of extra enemy fish so the harder difficulty is felt right away.

6.9 Game Restart — restartGame()

Resets score, lives, level, timers, player state, and clears both the food-fish and enemy-fish object pools so the game can be replayed without restarting the whole program.

7. Task Assignment and Codes of Functions

7.1 Task Assignment Table

Team Member	Assigned Task	Related Functions
Member 1	Background & environment (water, waves, bubbles, floor, grass, stones)	drawWater(), drawWave(), initializeBubbles(), updateBubbles(), drawBubbles(), drawSoil(), initializeGrass(), drawGrass(), drawStones()
Member 2	Colorful (food) fish visuals and scoring system	drawFoodFishLocal(), drawFoodFish(), spawnFoodFish(), updateFoodFish(), handleFoodCollisions()
Member 3	Player fish visuals, movement and keyboard interaction	drawPlayerFishLocal(), drawPlayerFish(), updatePlayerMovement(), specialKeyDown(), specialKeyUp(), keyboard()
Member 4	Enemy fish visuals, enemy movement, collision and difficulty levels	drawEnemyFishLocal(), drawEnemyFish(), spawnEnemyFish(), updateEnemyFish(), handleEnemyCollisions(), handlePlayerHit(), updateLevelFromScore()
All Members	HUD (score, lives, level), game loop, integration and testing	drawLives(), drawScore(), drawLevel(), drawGameOverOverlay(), timer(), display(), restartGame(), main()

7.2 Explanation of Key Integrated Functions

display() — Main Rendering Pipeline

This function is called every frame by GLUT and draws every object in the correct order: background first, then gameplay objects (fish), then foreground floor decorations, and finally the HUD on top of everything.

```
void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glLoadIdentity();

    drawWater(); drawWave(); drawBubbles();
    drawAllFoodFish(); drawAllEnemyFish(); drawPlayerFish();
    drawSoil(); drawGrass(); drawStones();

    drawLives(); drawScore(); drawLevel();
    drawControlHint(); drawLevelMessage(); drawGameOverOverlay();

    glutSwapBuffers();
}
```

timer() — Game Loop / Animation Driver

Instead of assuming a fixed frame time, this function calculates the actual elapsed time (deltaTime) between frames using glutGet(GLUT_ELAPSED_TIME), clamps it to a safe range, and passes it to every update function. This keeps the animation and physics speed consistent even if the frame rate slightly varies.

```
void timer(int) {
    int now = glutGet(GLUT_ELAPSED_TIME);
    float deltaTime = static_cast<float>(now - lastUpdateMilliseconds) / 1000.0f;
    lastUpdateMilliseconds = now;
    deltaTime = clampFloat(deltaTime, MIN_DELTA_TIME, MAX_DELTA_TIME);

    gameTime += deltaTime;
    updateBubbles(deltaTime);
    if (!gameState.gameOver) updateGameplay(deltaTime);

    glutPostRedisplay();
    glutTimerFunc(TIMER_INTERVAL_MS, timer, 0);
}
```

main() — Program Entry Point / OpenGL and GLUT Setup

Initializes GLUT, creates the window, registers every callback (display, reshape, keyboard, special keys, timer), and starts the main event loop.

```
int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(WIDTH, HEIGHT);
    glutCreateWindow("Fish Survival Game");

    initializeOpenGL();
```

```
    initializeGame();

    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutKeyboardFunc(keyboard);
    glutSpecialFunc(specialKeyDown);
    glutSpecialUpFunc(specialKeyUp);
    glutTimerFunc(TIMER_INTERVAL_MS, timer, 0);

    glutMainLoop();
    return 0;
}
```

8. Conclusion

Building the Fish Survival Game gave our team a practical, hands-on understanding of how the computer graphics concepts we learned in class actually come together to create a finished, interactive application. We successfully implemented an animated aquarium background, a keyboard-controlled player fish with smooth acceleration-based movement, colourful food fish with a scoring system, and dangerous enemy fish with a life-based penalty system, all rendered purely with basic OpenGL primitives no external images or sprites were used.

Through this project we gained direct experience with 2D shape construction using polygons and triangles, colour interpolation across vertices, 2D transformations (translation and scaling) for positioning and flipping objects, time-based animation using delta time, alpha blending for overlay screens, and simple but effective AABB collision detection. Just as importantly, we practiced dividing a real codebase into clear feature areas (background, player, food fish, enemy fish) so that each team member could work independently before integrating everyone's work into a single, working project a process that also required careful testing to make sure our individually written functions worked correctly together.

Given more time, the game could be extended in a few different directions:

- Adding more difficulty levels or a smoother difficulty curve.
- Introducing additional types of enemy fish with different movement patterns (e.g., fish that chase the player).
- Adding sound effects and background music for eating food, taking damage, and leveling up.
- Adding power-ups (e.g., temporary speed boost or shield).
- Improving fish animations with more detailed fin/tail movement or simple sprite-sheet-based swimming cycles.
- Adding a proper start menu with manual difficulty selection and a high-score leaderboard.